

Data Model: State, Memory, and Database

Overview

The simulator maintains three tiers of data:

- Live State -- the current quarter's world state (in memory)
- Run Memory -- per-agent rolling history within a single run
- Cross-Run Database -- persistent files accumulating across all runs

Tier 1: Live State

World State (maintained by Orchestrator)

```
@dataclass
class WorldState:
    # Time
    run_id: str
    quarter: int          # 1-indexed sequential quarter
    fyear: int            # fiscal year (2031, 2032, ...)
    fqtr: int             # fiscal quarter (1-4)

    # Macro
    macro: MacroState

    # Firms (indexed by firm_id)
    firms: dict[str, FirmState]
    firm_slots: dict[str, SlotInfo] # tracks incarnation, default history

    # Financial institutions
    institutions: dict[str, InstitutionState]

    # Environment
    environment_narrative: str
    active_events: list[Event]

    # Random state
    rng: numpy.random.Generator
```

MacroState

```
@dataclass
class MacroState:
    market_size_baseline: float # potential patients (before price filtering)
    risk_free_rate: float        # quarterly
    inflation_rate: float        # quarterly
    gdp_growth: float            # quarterly
    macro_shock: float           # demand multiplier shock
```

```

taste_shocks: dict[str, float] # per-firm demand shock
awareness_rate: float          # fraction of population aware of SRT
regulatory_mood: str           # "permissive", "stable", "tightening"

```

FirmState

```

@dataclass
class FirmState:
    firm_id: str
    incarnation: int
    is_active: bool
    quarters_alive: int

    # Balance Sheet -- Assets
    cash: float          # cheq
    accounts_receivable: float # rectq
    inventory_units: int
    inventory_value: float   # invtq
    ppe_gross: float        # ppegtq
    accumulated_depreciation: float
    ppe_net: float          # ppentq
    rd_capital: float       # intangible (if capitalizing R&D)
    other_assets: float

    # Balance Sheet -- Liabilities
    accounts_payable: float   # apq
    accrued_expenses: float   # acoq
    taxes_payable: float      # txpq
    revolver_balance: float    # dlcq (current portion of debt)
    long_term_debt: float     # dlттq
    other_liabilities: float

    # Balance Sheet -- Equity
    common_stock: float       # cstk
    apic: float               # additional paid-in capital
    retained_earnings: float  # req
    treasury_stock: float     # tstkq
    aoci: float               # accumulated other comprehensive income

    # Shares
    shares_outstanding: int

    # Derived (computed, not stored)
    @property
    def total_assets(self) -> float: ... # atq
    @property
    def total_liabilities(self) -> float: ... # ltq
    @property
    def total_equity(self) -> float: ... # ceqq
    @property
    def current_liabilities(self) -> float: ... # lctq = apq + acoq + txpq + dlcq

```

```

# Internal State (private to firm, not on financial statements)
capability_stock: float      # A_it -- R&D-driven quality index
brand_stock: float          # B_it -- marketing-driven brand index
capacity_units: int          # K_it -- max production per quarter
unit_cost: float             # current COGS per treatment course
product_generation: int      # 1, 2, 3, or 4
delivery_generation: int     # 1 (IV), 2 (subQ), 3 (oral), 4 (gene therapy)

# R&D Progress (private)
rd_cumulative_product: float
rd_cumulative_process: float
rd_cumulative_delivery: float

# Credit Terms (set by financial institutions)
revolver_commitment: float
revolver_rate: float
term_debt_rate: float
term_debt_max_new: float
equity_price: float

# Quality Scores (derived from generation + capability + delivery)
@property
def efficacy_score(self) -> float: ...
@property
def safety_score(self) -> float: ...
@property
def convenience_score(self) -> float: ...
@property
def quality_composite(self) -> float: ...

```

QuarterFlows (one per firm per quarter)

```

@dataclass
class QuarterFlows:
    firm_id: str
    quarter: int

    # Firm Decisions (raw LLM request)
    requested_price: float
    requested_production: int
    requested_capex: float
    requested_rd_spend: float
    requested_rd_allocation: dict[str, float]
    requested_sga_spend: float
    requested_equity_issuance: float
    requested_debt_issuance: float
    requested_dividends: float
    requested_buybacks: float

    # Actual (after clamping by orchestrator)
    actual_price: float
    actual_production: int

```

```

actual_capex: float
actual_rd_spend: float
actual_sga_spend: float
actual_dividends: float
actual_buybacks: float

# Market Outcomes (from environment agent)
units_sold: int
market_share: float

# Income Statement
net_sales: float          # saleq
cogs: float               # cogsq
gross_profit: float       # gpq
rd_expense: float         # xrdq
sga_expense: float        # xsgaq
depreciation: float       # dpq
operating_income: float   # oiadpq
interest_expense: float   # xintq
pretax_income: float      # piq
tax_expense: float        # txtq
net_income: float         # niq

# Cash Flow Statement
cfo: float                # oancfq
cfi: float                # ivncfq
cff: float                # fincfq
change_in_cash: float     # chechq

# Events
default_flag: bool
ipo_flag: bool
clinical_hold_flag: bool

```

SlotInfo (per firm slot, persists across incarnations)

```

@dataclass
class SlotInfo:
    slot_id: str          # "slot_0" through "slot_4"
    current_firm_id: str   # "firm_0_inc_1"
    incarnation: int
    consecutive_q1_defaults: int
    total_defaults: int
    is_frozen: bool
    is_paused: bool       # paused for 1 quarter after N consecutive defaults
    default_history: list[dict] # [{incarnation, default_quarter, lifespan}]

```

InstitutionState

```

@dataclass
class InstitutionState:
    institution_id: str

```

```

institution_type: str          # "investment_bank", "commercial_bank", "credit_fund"

# Portfolio
loans_outstanding: dict[str, LoanPosition] # firm_id -> position
equity_positions: dict[str, EquityPosition]

# Performance tracking
total_principal_advanced: float
total_principal_recovered: float
total_interest_earned: float
total_losses: float
defaults_experienced: int

```

Event (active events affecting the simulation)

```

@dataclass
class Event:
    event_type: str          # "safety_scandal", "regulatory_change", "breakthrough", etc.
    description: str
    affected_firms: list[str] # empty = all firms
    start_quarter: int
    duration_quarters: int
    demand_impact: float     # multiplier (e.g., -0.20 = 20% demand reduction)
    cost_impact: float       # multiplier on COGS
    regulatory_impact: str   # "clinical_hold", "pricing_cap", "none"
    is_active: bool

```

Tier 2: Run Memory

MarketMemory (public, shared across all agents)

A chronological log of public events, one entry per quarter:

```

@dataclass
class MarketMemoryEntry:
    quarter: int
    fyear: int
    fqtr: int

    # Macro
    macro_summary: str          # "Market growing at 3%, risk-free rate stable at 4%"

    # Firm public actions
    firm_actions: list[dict]    # [{firm_id, price, market_share, revenue, ...}]

    # Financial terms
    equity_prices: dict[str, float]
    credit_terms_summary: str

    # Events
    events: list[str]

```

```
# Environment narrative
narrative: str
```

FirmMemory (private, one per firm)

```
@dataclass
class FirmMemoryEntry:
    quarter: int

    # State snapshot (private)
    cash: float
    total_assets: float
    total_debt: float
    capability_stock: float
    brand_stock: float
    capacity: int
    unit_cost: float
    product_generation: int
    rd_progress: dict[str, float]

    # Decisions made
    decisions: dict          # full decision dict

    # Outcomes
    units_sold: int
    revenue: float
    net_income: float
    market_share: float

    # Internal KPIs
    gross_margin: float
    cash_burn_rate: float
    quarters_of_cash_remaining: float
    rd_efficiency: float      # capability gain per dollar
```

InstitutionMemory (private, one per institution)

```
@dataclass
class InstitutionMemoryEntry:
    quarter: int

    # Portfolio snapshot
    total_exposure: float
    exposure_by_firm: dict[str, float]

    # Decisions made
    terms_offered: dict[str, dict]

    # Outcomes
    interest_collected: float
    principal_repaid: float
```

```
losses_incurred: float
portfolio_return: float
```

Memory Management

- Short-term memory: Last K quarters (default K=4) -- full detail
- Long-term memory: Compressed summary of earlier quarters
 - Generated by the orchestrator (not by the agent's own LLM -- avoids hallucination)
 - Format: "Over quarters 1-8, [agent] pursued [strategy], achieving [outcomes]..."
- Memory is included in each prompt as structured context

Tier 3: Cross-Run Database

Compustat Panel (data/compustat_q.csv)

Append-only CSV with one row per firm-quarter. Primary key: (run_id, firm_id, incarnation, fyearq, fqtr).

Column	Compustat Name	Description
run_id	--	Simulation run identifier
firm_id	gvkey	Firm identifier
incarnation	--	Incarnation counter for the slot
fyearq	fyearq	Fiscal year
fqtr	fqtr	Fiscal quarter (1-4)
saleq	saleq	Net sales / revenue
cogsq	cogsq	Cost of goods sold
gpq	gpq	Gross profit
xrdq	xrdq	R&D expense
xsgaq	xsgaq	SGA expense
dpq	dpq	Depreciation
oiadpq	oiadpq	Operating income
xintq	xintq	Interest expense
piq	piq	Pretax income
txtq	txtq	Tax expense
niq	niq	Net income
cheq	cheq	Cash and equivalents
rectq	rectq	Accounts receivable
invqtq	invqtq	Inventories
ppentq	ppentq	PP&E net
atq	atq	Total assets
apq	apq	Accounts payable
acoq	acoq	Accrued expenses
txpq	txpq	Taxes payable
dlcq	dlcq	Current debt (revolver)
lctq	lctq	Total current liabilities
dlttq	dlttq	Long-term debt
ltq	ltq	Total liabilities
cstkq	cstkq	Common stock
ceqq	ceqq	Total common equity
req	req	Retained earnings
sstkq	sstkq	Stock issuance this quarter

prstkq	prstkq	Stock repurchases this quarter
dvq	dvq	Dividends this quarter
oancfq	oancfq	Cash from operations
ivncfq	ivncfq	Cash from investing
fincfq	fincfq	Cash from financing
chechq	chechq	Change in cash
capxq	capxq	Capital expenditure
prccq	prccq	Equity price per share
cshoq	cshoq	Shares outstanding
mkvaltq	mkvaltq	Market cap
default_flag	--	1 if firm defaulted this quarter
pricing_error	--	P_it - P_star_it (equity mispricing)

Debrief (data/debrief.csv)

Append-only CSV with performance metrics per firm-incarnation and per institution.

Column	Description
run_id	Run identifier
actor_id	firm_0, ibank_0, cbank_0, cfund_0
actor_type	firm, investment_bank, commercial_bank, credit_fund
incarnation	For firms; 0 for institutions
lifespan_quarters	Quarters active
total_revenue	Lifetime revenue
total_equity_issued	Total equity raised
total_debt_issued	Total debt raised
total_interest_paid	Lifetime interest expense
total_dividends	Lifetime dividends paid
total_buybacks	Lifetime buybacks
terminal_equity_value	P_T * shares if alive; liquidation residual if defa
equity_irr_quarterly	Quarterly IRR for equity investors
equity_irr_annualized	Annualized IRR
equity_total_return	Money-weighted total return
debt_total_return	For institutions: total return on lending
debt_irr_annualized	For institutions
loss_rate	1 - recovered / advanced
default_flag	Whether firm defaulted
default_quarter	Quarter of default (if applicable)
max_generation_achieved	Highest product generation reached
avg_market_share	Average market share over life
pricing_rmse	For investment bank: RMSE of equity pricing errors
pricing_bias	For investment bank: average signed pricing error

Past Simulation Retrieval

When building prompts, the orchestrator can query the cross-run database:

```
def retrieve_similar_cases(
    current_firm_state: FirmState,
    database_path: str,
    k: int = 3
) -> list[dict]:
    """
```



```
Find k most similar firm-quarters from past runs.
Similarity based on: total_assets, revenue, debt/equity, product_generation, market_share.
Returns summary dicts suitable for prompt inclusion.
"""
```

This gives agents "institutional knowledge" from past simulations -- e.g., "firms in your situation (Gen 1, \$300M assets, 20% market share) that invested heavily in R&D typically achieved Gen 2 within 8 quarters and saw 3x revenue growth."

Validation Invariants

The orchestrator checks after every quarter:

Hard Invariants (must pass or run is invalid)

- $atq \geq 0$ for all non-default rows
- $cheq \geq 0$ for all non-default rows
- $|atq - ltq - ceqq| < 1.0$ (balance sheet identity)
- $|lctq - (apq + acoq + txpq + dlq)| < 1.0$ (current liabilities decomposition)
- $|chechq - (cheq_t - cheq_{t-1})| < 1.0$ (cash reconciliation)
- No duplicate (run_id, firm_id, incarnation, fyearq, fqtr) keys
- $units_sold \leq production$ for all firms
- $sum(market_shares) \approx 1.0$ (within tolerance)

Soft Invariants (flagged as warnings)

- No firm has >50% market share for >4 consecutive quarters
 - R&D spending is non-zero for at least some firms
 - Equity prices are positive and within reasonable range
 - Cash flow statement ties: $chechq == oancfq + ivncfq + fincfq$ (within tolerance)
 - Retained earnings roll-forward: $RE_t = RE_{t-1} + NI_t - DIV_t$
-

File Layout

```
LLM_firms/
  docs/
    world/          # World-building documents (7 files + PDFs)
    architecture/   # These architecture documents
  data/             # Cross-run persistent data
    compustat_q.csv
    debrief.csv
  outputs/          # Per-run outputs
    {run_id}/
      compustat_q.csv
      debrief.csv
      statements/
        firm_0_Q1_2031.txt
        ...
      checkpoints/
        quarter_01.pkl
```

```
...
logs/
  orchestrator.log
  env_0.log
  firm_0.log
...
config/
  default.yaml      # Default configuration
src/                # Source code (Python package)
  orchestrator/
  agents/
  accounting/
  ...
```